CS 313 Software Engineering 1 · Weeks 7 and 8

From Game Idea to Blueprint

Library Game Hub: Architecture, REST API Design, Design Patterns, and the Alpha Design Check

Ailen S. Laguda, LPT, MIS
BSCS 3-1 and 3-2 · A.Y. 2026–2027

Where we are

Done	**Weeks 1 to 6:** Requirements, use cases, class diagram, sequence and activity diagrams, ERD
Last week	**Week 7:** Capstone functional testing (so we are one week behind)
This week	**Weeks 7 + 8:** Architecture and REST API, then the Alpha Design Check
Next week	**Week 9:** Midterm Exam and Sprint 1 Demo

Our plan for this week: 5 hours

Lecture (2 hours)

Time	Topic
0:00–0:10	Where we are + lessons from testing
0:10–0:30	Game concept and the Library Game Hub
0:30–0:50	Software architecture
0:50–1:25	REST API design
1:25–1:45	Design patterns + diagrams
1:45–2:00	Alpha Design Check briefing

Lab (3 hours)

Time	Task
0:00–1:00	Build: Game Concept, API table, diagrams
1:00–1:40	Peer review in pairs
1:40–2:40	Revise all documents
2:40–3:00	Export PDF and submit

Most bugs we found started in the design

The bug we saw	Where it really started
A form accepts empty or wrong input	No validation rules in the requirements or API design
Data is saved wrong, or goes missing	Weak ERD: missing fields, keys, or relationships
A user sees a page they should not see	Roles and access were never planned in the use cases

A mistake found on paper takes minutes to fix. The same mistake found in code takes days.

Part 1: Game Concept and the Library Game Hub

Your midterm project: one game mode that plugs into the library’s main game page

How the Library Game Hub works

1. A host (a librarian or teacher) logs in.
2. The host picks a question set, for example “Philippine Literature.”
3. The host picks a game mode from the main page.
4. The server gives a 6-digit join code, for example 482913.
5. Players join with a nickname and the code. No account needed.
6. Everyone plays, then sees the leaderboard.

It works like Blooket: same question sets, many different game modes.

Who builds what, and when

Part	When	Who
Your game's REST API (backend)	THIS semester	Each team
The Hub Contract (the shared rules)	THIS week	Instructor
The hub main page	Next semester	Instructor or a volunteer team
Your game's screens (frontend)	Next semester	Each team

This semester, your goal is simple: a working, tested API that follows the Hub Contract.

Our sample game mode: Shelf Climb

“Answer questions to climb the library bookshelf. First to the top shelf wins.”

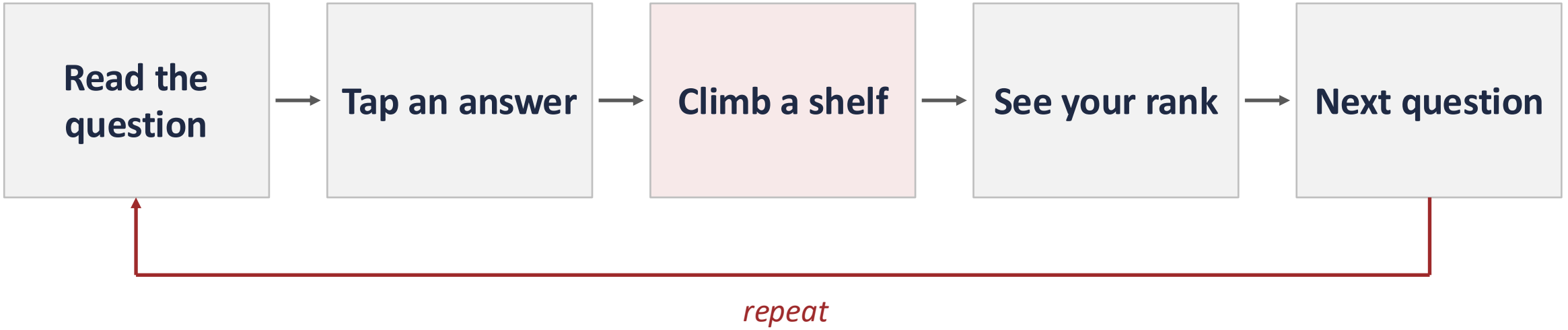
Part	Shelf Climb
Genre and platform	Multiplayer quiz race in a web browser
Players	Library visitors: high school and college students
How to play	Tap the correct answer before the timer ends
Win or lose	Each correct answer climbs 1 shelf. First to shelf 10 wins.
Backend features	Host login, question sets, sessions, join codes, answers, leaderboard

This is only a sample. Your team makes its own game mode.

The 8 parts of a Game Concept Document

1. Title and one-line pitch
2. Genre and platform
3. Target players
4. Core loop
5. Mechanics and controls
6. Win or lose rules
7. Feature list (MoSCoW)
8. **Backend features and Hub Contract fit**

The core loop: what the player repeats



Every game uses the same questions. Your core loop is what makes your game different.

One feature, followed through every document

Document	Example: “Join a game with a code”
Game Concept	Players join with a 6-digit code
User story (Week 2)	As a player, I want to join with a code so that I can play without an account.
Use case (Week 3)	UC-05 Join Session
Class (Week 4)	SessionPlayer: id, sessionId, nickname, score
Table (Week 6)	session_players: id (PK), session_id (FK), nickname, score
API endpoint (today)	`POST /api/v1/sessions/{code}/players`

If a Must-have feature is missing from any row, there is a gap. Gaps become bugs.

Part 2: Software Architecture

The big decisions about how your system is built

Architecture is the blueprint of your system

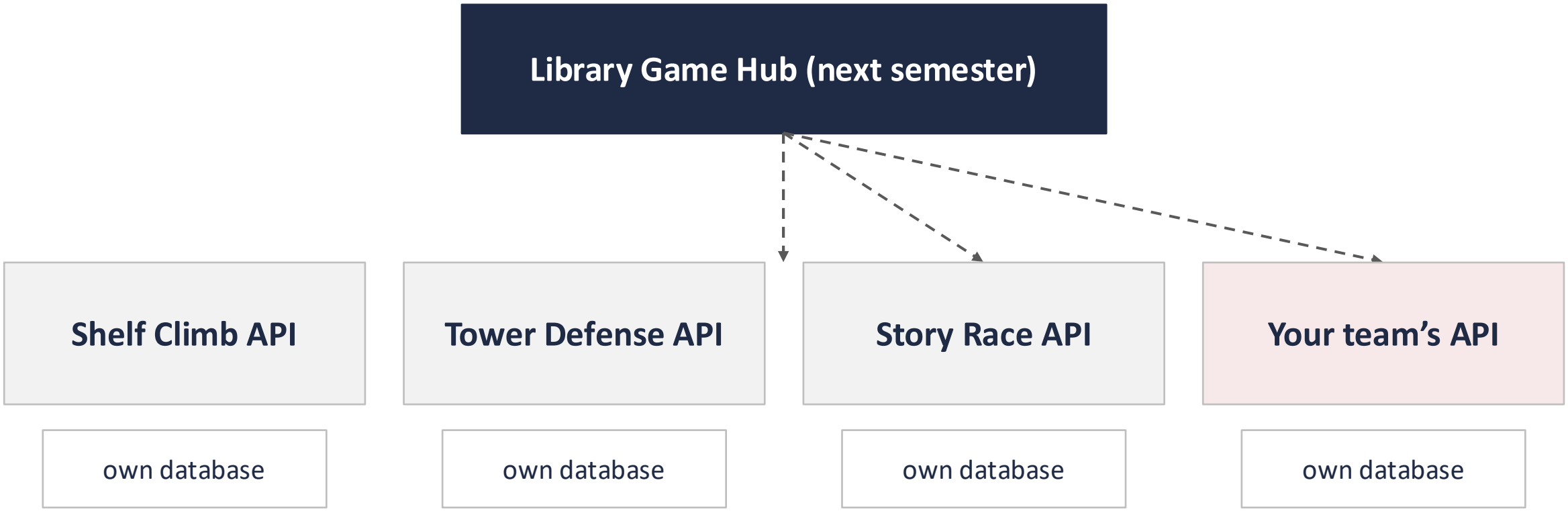
Before building a house, the architect decides the rooms, how they connect, and where the pipes go. Moving a wall later is expensive. Software is the same.

Question	Answered by	Shelf Climb example
What are the parts?	Component diagram	Auth, Question Sets, Sessions, Answers
How do the parts talk?	API design	HTTP requests with JSON data
Where does each part run?	Deployment diagram	Library server and database

Four architecture styles you must know

Style	Meaning	Example
Monolithic	Everything in one big program	One team puts the hub and every game in one app
Layered (MVC)	Code is split by job into layers	Routes, controllers, services, repositories
Client-server	The client asks, the server answers	Postman asks your API for a question set
RESTful	Client-server over HTTP, with resources and JSON	GET /api/v1/question-sets/12

Our design: one hub, many game APIs



The dashed lines are the Hub Contract.

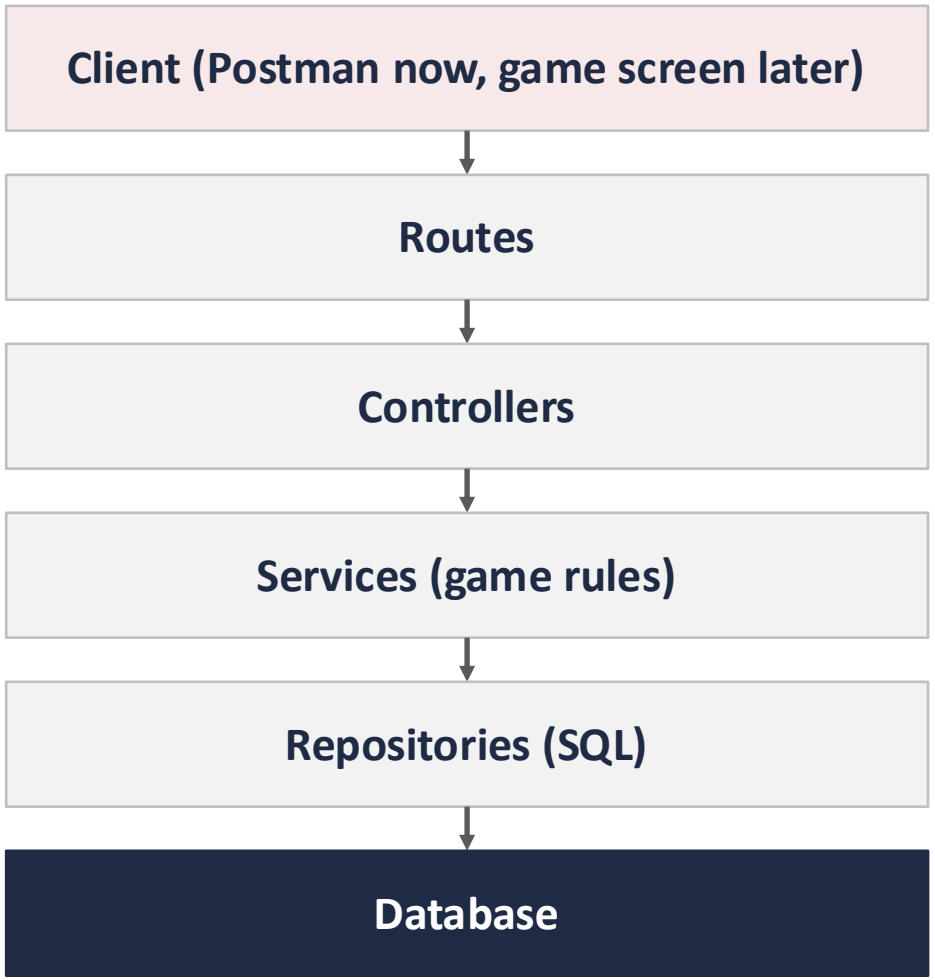
Like a wall socket: any appliance works if its plug has the right shape.

MVC: each part has only one job

Part	Its job	Example
Model	The data and its rules	A question must have exactly one correct answer.
View	What goes back to the user	In an API, the View is the JSON response.
Controller	The traffic officer	Takes the request, asks the Model, sends the reply.

Like a restaurant: you order, the waiter (Controller) goes to the kitchen (Model), and your plate (View) comes back.

One request, layer by layer



Example: a player answers a question

1. The client sends POST /sessions/482913/answers
2. The route sends it to the controller
3. The controller checks the input
4. The service checks the rules: is the game open? Is the answer correct?
5. The repository saves the answer
6. The reply: 201 Created, correct: true

What the client does, and what the server does

Game screen (next semester)

- Graphics and sounds
- The timer on screen
- Showing the question

Your API (this semester)

- Host login
- Question sets and sessions
- Checking if an answer is correct
- Scores and leaderboard

Golden rule: never send the correct answer to the player. Anyone can press F12 and read it.

Part 3: REST API Design

How a client and your server talk to each other

An API is like a waiter



	Plain words	API words
Request	“Show me the ranking.”	`GET /api/v1/sessions/482913/leaderboard`
Response	“Here it is.”	`200 OK + the ranking in JSON`

The customer never walks into the kitchen.

REST in 4 simple rules

Rule	Example
1. URLs are nouns (things), not actions	`/question-sets  /sessions`
2. HTTP verbs are the actions	`POST /sessions  (start a new game)`
3. Stateless: every request carries its token	`Authorization: Bearer eyJhbGci...`
4. Data is sent as JSON	`{ "nickname": "Bookworm" }`

The 5 HTTP verbs

Verb	Meaning	Example
GET	Read	`GET /question-sets/12`
POST	Create	`POST /question-sets`
PUT	Replace everything	`PUT /question-sets/12`
PATCH	Change a part	`PATCH /question-sets/12`
DELETE	Remove	`DELETE /question-sets/12`

Easy to remember: they match CRUD. Create, Read, Update, Delete.

Status codes: the server’s short answer

Code	Meaning	Example
200	OK	The leaderboard loaded
201	Created	The player joined the game
400	Bad Request	The nickname is empty
401	Unauthorized	The host is not logged in
403	Forbidden	A host edits another host’s question set
404	Not Found	Join code 999999 does not exist
409	Conflict	The nickname is already taken
500	Server Error	A bug crashed the server

2xx = success 4xx = the client’s mistake 5xx = the server’s mistake

A real request and response

Request (client to server)

```
POST /api/v1/sessions/482913/answers
Authorization: Bearer <player token>

{
  "question_id": 3,
  "choice_index": 1
}
```

Response: 201 Created

```
{
  "correct": true,
  "score": 700,
  "shelf": 7,
  "rank": 2
}
```

The player sends a CHOICE, not “I am correct.” The server decides.

Good URLs and bad URLs

Bad	Good
`/getAllQuestionSets`	`GET /question-sets`
`/createSession`	`POST /sessions`
`/questionSet/delete?id=12`	`DELETE /question-sets/12`
`/getPlayers?code=482913`	`GET /sessions/482913/players`
`/Question_Sets`	`/question-sets`

Rules: plural nouns, lowercase, hyphens, no verbs, start with /api/v1

The Hub Contract: 6 rules every team follows

Rule	What it means
`1. GET /api/v1/game-info`	Returns your game's name, description, and details
`2. GET /api/v1/health`	200 if the API and database work, 503 if not
3. One question-set format	Same JSON for all teams; never send the correct answer
4. One error format	Same error JSON and status codes for all teams
5. Sessions and join codes	6-digit codes; players give a nickname only
6. URL rules	/api/v1, plural nouns, lowercase, hyphens

Full details are in the Hub Contract v1 handout.

Example: what game-info returns

GET /api/v1/game-info returns 200 OK

```
{
  "slug": "shelf-climb",
  "name": "Shelf Climb",
  "team": "Bookworms (3-1)",
  "description": "Climb the shelf
     by answering questions.",
  "min_players": 1,
  "max_players": 30,
  "version": "1.0.0"
}
```

- No login needed
- Easy first endpoint
- The hub will read this next semester

Your API Endpoint Table (sample)

Verb	URL (after /api/v1)	What it does	Token	OK	Errors
GET	`/game-info`	Hub reads game details	No	200	500
POST	`/auth/login`	Host logs in	No	200	400, 401
POST	`/question-sets`	Create a question set	Host	201	400, 401
POST	`/sessions`	Start a game	Host	201	400, 401, 404
POST	`/sessions/{code}/players`	Player joins	No	201	400, 404, 409
POST	`/sessions/{code}/answers`	Submit an answer	Player	201	400, 401, 409
GET	`/sessions/{code}/leaderboard`	See the ranking	No	200	404

Your table also needs a request body column and a use case column.

Part 4: Design Patterns and Diagrams

Tested recipes, plus the component and deployment diagrams

Design patterns: tested recipes

Pattern	In one sentence	In our project
Singleton	Only one of it exists.	One database connection pool for the whole API
Factory	One place makes the objects.	Makes a true/false or multiple choice question
Repository	The only door to the data.	All SQL lives here; controllers never write SQL

The Repository pattern in code

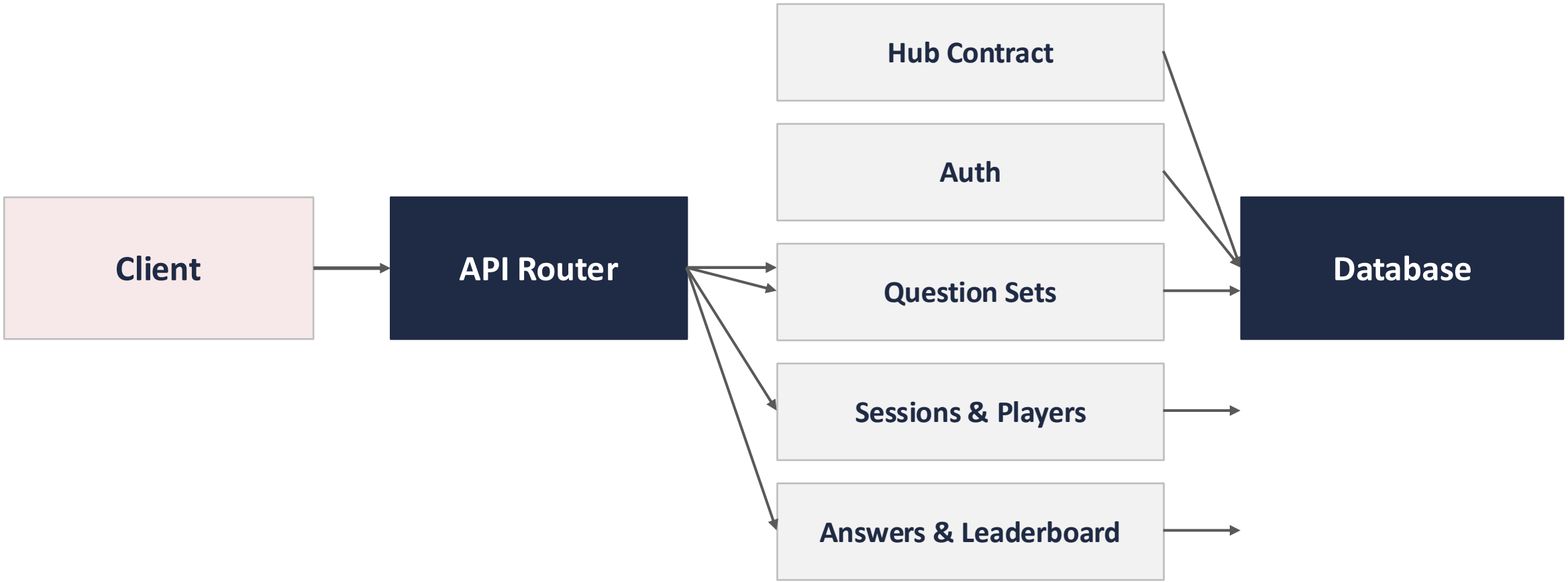
playerRepository.js: the only place with SQL

```
function getLeaderboard(sessionId) {
  return db.query("SELECT nickname, score FROM session_players
    WHERE session_id = ? ORDER BY score DESC", [sessionId]);
}
```

leaderboardController.js: no SQL here

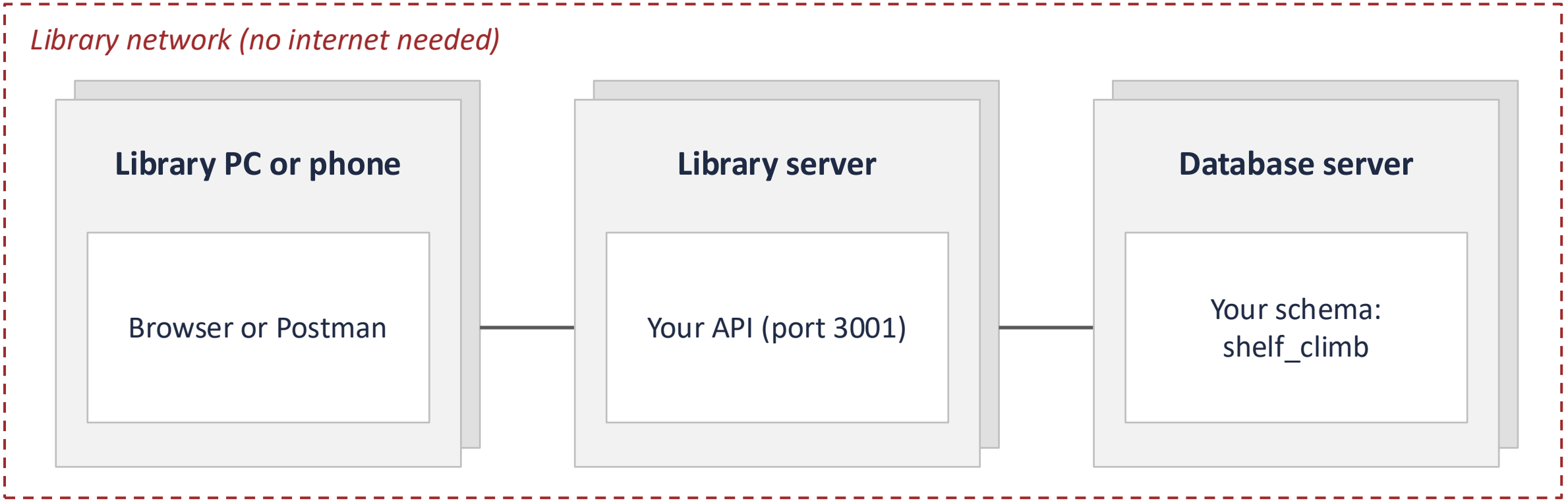
```
async function show(req, res) {
  const top = await playerRepository.getLeaderboard(req.session.id);
  res.status(200).json(top);
}
```

Component diagram: the parts of your API



One component per group of endpoints. Use the same names as your class diagram.

Deployment diagram: where it runs in the library



Boxes are machines (nodes). Inside is what runs on them. Lines are connections.

Part 5: Alpha Design Check

A friendly inspection before the Week 9 Sprint 1 Demo

Your design set: 8 documents in one PDF

1. **Game Concept Document (finish today)**
2. User Story Backlog with MoSCoW
3. Use Case Diagram + 3 narratives
4. Class Diagram
5. Sequence + Activity Diagrams
6. ERD + Schema + Data Dictionary
7. **API Endpoint Table (new)**
8. **Architecture, Component, Deployment Diagrams (new)**

What reviewers check: do your documents agree?

- Every Must-have user story has a use case.
- Every class that stores data has a table.
- Every use case has an API endpoint.
- game-info and health are in the API table.
- The same names are used everywhere.

Example gap: the class diagram has a Choice class, but the ERD has no choices table. The server cannot check answers.

How the peer review works

1. Two teams sit together.
2. Team A presents its draft (8 minutes).
3. Team B checks it with the checklist and traces one feature (10 minutes).
4. Switch roles.
5. Each team writes its top 3 fixes.

Give feedback as 1 Glow (something good) and 2 Grows (specific things to fix).

Common mistakes and how to fix them

Mistake	Fix
Verbs in the URL: /getQuestions	GET /question-sets
No error codes in the API table	Add 400, 401, 404, 409 where they apply
ERD tables without foreign keys	Add a PK and FK to every relationship
“Play Game” as one use case	Split: Join Session, Submit Answer, View Rank
Correct answer sent to the player	Send choices only; the server checks
Session, Game, and Room used for one thing	Pick one name and use it everywhere

Lab plan: 3 hours

Time	Task	Who
0:00–1:00	Finish the Game Concept Document	Product Owner
	Make the API Endpoint Table (with Hub Contract)	Developers
	Draw the architecture, component, deployment diagrams	QA Lead + Tech Writer
	Update Trello and compile the draft	Scrum Master
1:00–1:40	Peer review in pairs	Whole team
1:40–2:40	Fix the top 3 issues, then check names	Whole team
2:40–3:00	Export PDF, submit, stand-up	Scrum Master

Before you leave the lab

1. One PDF named CS313_[Section]_[TeamName]_DesignSet_v1.pdf
2. Documents in order, with a cover page and table of contents
3. Uploaded to InsightEd before the deadline
4. PDF and diagram source files pushed to GitHub /docs
5. Trello cards updated

Next week: Midterm Exam and Sprint 1 Demo

Lecture: Midterm Exam

- Weeks 1 to 8, including today
- UML, ERD, architecture
- REST, verbs, status codes
- Design patterns

Lab: Sprint 1 Demo

- Final design PDF
- Game Concept
- API project set up, database connected
- Working in Postman: game-info and health

Study tip: explain how one feature of your game goes from user story to API endpoint.

Questions?

Next: the lab. Sit with your team and open the lab plan.

Handout: Game Concept template

Part	Your team’s answer
1. Title and one-line pitch	
2. Genre and platform	
3. Target players	
4. Core loop (4 to 6 verbs)	
5. Mechanics and controls	
6. Win or lose rules	
7. Features (Must, Should, Could, Won’t)	
8. Backend features and Hub Contract fit	

Handout: peer review checklist

Check	Yes	No	Notes
Game Concept has all 8 parts and a clear core loop			
Every Must-have story has a use case			
Class diagram shows multiplicity (1, *)			
Every table has a PK; relationships have FKs			
Every use case has an API endpoint			
game-info and health are in the API table			
Question sets follow the contract; no answer sent to players			
Every endpoint lists success and error codes			
URLs follow the rules			
Names match in ALL documents			

Feature we traced: _____ Glow: _____ Grow: _____